

BÁO CÁO ĐỒ ÁN MÔN HỌC

(Đồ án phát triển ứng dụng)

Lớp: IT003.Q21.CTTN

SINH VIÊN THỰC HIỆN

Mã sinh viên: 25521787

Họ và tên: Võ Quốc Thịnh

TÊN ĐỀ TÀI: Game Pacman

CÁC NỘI DUNG CẦN BÁO CÁO:

1. Giới thiệu đồ án:

a. Mô tả chung về ứng dụng:

Dự án là một phiên bản mô phỏng (clone) của tựa game arcade kinh điển Pac-Man. Người chơi điều khiển nhân vật chính để thu thập các hạt đậu trong mê cung.

b. Các CTDL và giải thuật đã được sử dụng: (cần làm rõ các câu hỏi: what, why)

- Mảng 2D (2D Array / Ma trận):

- What:** Cấu trúc dữ liệu dùng để lưu trữ và biểu diễn cấu trúc của bản đồ mê cung. Mỗi phần tử trong ma trận tương ứng với một trạng thái của ô
- Why:** Mảng 2D cho phép truy xuất tọa độ ngẫu nhiên trong thời gian $O(1)$. Điều này giúp thao tác kiểm tra va chạm (collision detection) diễn ra nhanh chóng, tối ưu hóa hiệu suất thay vì tính toán hình học phức tạp.

- Hàng đợi (Queue)

- What:** Cấu trúc dữ liệu hoạt động theo nguyên tắc **FIFO (First-In-First-Out)**, được khai báo trong

hàm `bfs_shortest_path()` tại `algorithms.py` dưới dạng `queue = collections.deque([start])`. Dùng để lưu trữ các đỉnh (tọa độ lưới) đang chờ được duyệt.

2. **Why:** Hàng đợi đảm bảo thuật toán BFS duyệt các ô theo từng cấp độ — tất cả các ô cách start đúng k bước được duyệt xong trước khi duyệt các ô cách k+1 bước. Thao tác `popleft()` và `append()` trên deque đều có thời gian $O(1)$, giúp BFS hoạt động hiệu quả

- **Giải thuật Tìm kiếm theo chiều rộng (Breadth-First Search - BFS):**

1. **What:** Thuật toán duyệt đồ thị lưới trong hàm `bfs_shortest_path(start, target, grid)` tại `algorithms.py`. Bắt đầu từ vị trí Ghost, BFS lần lượt: (1) lấy đỉnh đầu hàng đợi, (2) kiểm tra có phải đích không, (3) thêm tất cả đỉnh kề chưa duyệt vào hàng đợi, (4) lặp lại cho đến khi tìm thấy đích hoặc hết đỉnh.
2. **Why:** Trong mê cung Pac-Man, mọi bước di chuyển đều có chi phí bằng 1 (đồ thị không trọng số). BFS **đảm bảo** tìm được đường đi ngắn nhất, giúp Ghost luôn chọn hướng tối ưu nhất để tiếp cận Pac-Man.

2. Quá trình thực hiện

- a. Tuần 1: Khởi tạo hệ thống, Mảng 2D và Cơ chế di chuyển :
 - Thiết lập môi trường: Khởi tạo dự án với thư viện pygame và xây dựng vòng lặp trò chơi (Game Loop).
 - Ứng dụng Mảng 2D biểu diễn bản đồ: Tải ma trận 2 chiều để lưu trữ cấu trúc bản đồ. Xây dựng hàm render để duyệt mảng và hiển thị các khối hình học lên màn hình.

- Xử lý Logic di chuyển & Va chạm $O(1)$: Nhận tín hiệu điều hướng từ người chơi. Tọa độ của Pac-Man được biểu diễn trực tiếp trên ma trận. Nếu giá trị tại ô đó là tường ('1'), nhân vật bị chặn lại.

- Tương tác cơ bản: Cập nhật giá trị ô ma trận từ hạt đậu ('0') thành đường trống (' ') khi nhân vật đi ngang qua.

b. Tuần 2: Xây dựng AI tìm đường (BFS) và script demo minh họa:

3. Kết quả đạt được

- Thiết lập thành công hệ thống hiển thị bản đồ dựa trên Mảng 2D.
- Nhân vật thao tác mượt mà trên lưới đồ họa, tương tác vật lý hoạt động chính xác.
- Xây dựng được nền tảng chuẩn bị cho việc tích hợp AI tìm đường vào tuần sau.

4. Tài liệu tham khảo

- Giáo trình Cấu trúc dữ liệu và Giải thuật.
- Tài liệu lập trình thư viện Pygame.

5. Phụ lục 1: Giới thiệu (demo) kết quả

<https://youtu.be/32tAwuCp5Kg>

6. Phụ lục 2: docstring

- Module Docstring:

```
"""
author : vqthinh
WEEK 1 DEMO: Basic 2D Arrays, Map Rendering, and Movement.
This script demonstrates the fundamental concept of treating
a 2D matrix
as a game world grid. It features a simplified Pacman with
basic wall
collision.
"""
```

- Class BasicPacman:

```
""  
  
author : vqthinh  
  
A simplified version of the Pacman entity for the initial  
week 1 demonstration.  
  
Focuses on discrete movement within a 2D grid matrix.  
  
""
```

- Hàm update (Collision & Movement):

```
""  
  
author : vqthinh  
  
Updates Pacman's position based on queued input and grid  
boundaries.  
  
Demonstrates O(1) wall collision by indexing directly into  
the 2D grid array.  
  
Args: grid (list[list[str]]): The grid matrix being  
navigated.  
  
""
```

- Hàm draw:

```
"""  
  
author : vqthinh  
  
Draw a simple yellow circle to stands for Pacman  
  
"""
```

- Hàm draw_grid:

```
"""  
  
author : vqthinh  
  
Renders the game map from the 2D matrix.  
  
Args:  
  
surface (pygame.Surface): The Pygame display surface.  
  
grid (list[list[str]]): The 2D array representation of the  
map.  
  
"""
```

- Hàm bfs-shortest_path:

```
"""  
  
HARD DIFFICULTY AI (Breadth-First Search):  
Logic: Level-order traversal to find the shortest path in an  
unweighted graph.
```

Guarantees the absolute shortest path to the target (Pacman) by exploring all possible paths level-by-level using a Queue.

Data Structures:

- Queue (collections.deque): For $O(1)$ enqueue/dequeue operations.
- Dictionary (parent_map): To track visited nodes and reconstruct the path.

Args:

start (tuple): The starting (row, col) position.

target (tuple): The target (row, col) position (usually Pacman).

grid (list[list[str]]): The 2D grid matrix.

occupied_positions (set, optional): Positions of other ghosts.

Returns:

tuple: The immediate next (row, col) position on the shortest path.

Complexity:

Time: $O(V + E)$ where V is the number of grid cells and E is the number of valid moves.

Space: $O(V)$ to store the visited nodes in the parent_map and queue.

"""

- **Hàm random_walk_algorithm:**

```
"""
EASY DIFFICULTY AI / BFS FALLBACK:
Logic: Random selection of valid neighbors.

This algorithm picks a random valid direction at each step.
In demo_tuan2.py, it only activates as fallback when BFS
cannot find a path to the target (target fully enclosed).

Args:
    start (tuple): The current (row, col) position of the
ghost.
    grid (list[list[str]]): The 2D grid matrix.
    occupied_positions (set, optional): Positions of other
ghosts.

Returns:
    tuple: The next (row, col) position to move to.

Complexity:
    Time: O(1)
"""
```

- **Hàm get_valid_neighbors:**

```
"""
Finds all navigable adjacent cells (Up, Down, Left, Right)
from a given position.
```

A cell is considered navigable if it is within grid boundaries, not a wall ('1'), and not currently occupied by another entity. Includes Tunnel Wrap Around logic.

Args:

pos (tuple): The current (row, col) position.

grid (list[list[str]]): The 2D grid matrix.

occupied_positions (set, optional): Positions of other ghosts to avoid collision.

Returns:

list[tuple]: A list of navigable (row, col) neighbor positions.

Complexity:

Time: $O(1)$ - Constant time as it only checks 4 directions.

"""